

10-模拟专题

10-模拟专题

一、本阶段目标

模拟是 CSP 中非常高频的一类题型。

这一章的目标是：

- 理解模拟题到底在考什么
- 学会把题目规则翻译成代码
- 学会设计状态变量
- 减少边界、编号、输入格式错误
- 掌握一维模拟、二维模拟、日期模拟、字符串模拟
- 能独立完成 CSP A/B 题中的常规模拟题

模拟题的核心不是算法难。

而是：

把题目规则准确、完整、稳定地写成代码。

二、什么是模拟？

模拟就是：

题目怎么说，程序就怎么做。

例如：

题目说每次向右走一步
代码就更新列坐标

题目说遇到边界就转向
代码就判断是否越界

题目说连续三个相同就消除
代码就检查连续段

模拟题本质是：

规则翻译题。

三、模拟题为什么容易错？

模拟题看起来简单，但很容易错在：

- 下标
- 边界
- 编号
- 输入格式
- 状态变量
- 更新顺序
- 特殊情况

尤其 CSP 里，模拟题经常不是难在思想，而是难在：

细节特别多。

四、模拟题的一般解题步骤

1. 读懂规则

先弄清楚：

题目中有哪些对象？
每个对象有哪些状态？
每一步发生什么变化？
什么时候结束？
输出什么？

2. 设计变量

例如：

当前位置：x, y
当前方向：dir
当前时间：t

当前计数：cnt
当前状态：flag

3. 按顺序写流程

不要一上来就写代码。

先在草稿里写：

输入
初始化
循环模拟
状态更新
输出

4. 检查边界

重点检查：

第一个
最后一个
空情况
只有一个元素
越界情况

五、一维模拟

1. 常见模型

一维模拟通常发生在：

数组
队列
字符串
编号序列
时间线

2. 典型问题

例如：

从左到右扫描数组
统计连续相同的元素
根据规则修改某些位置

3. 常用写法

```
for(int i=0;i<n;i++)  
{  
    // 按顺序处理第 i 个元素  
}
```

4. 连续段统计模板

```
int cnt=1;  
  
for(int i=1;i<n;i++)  
{  
    if(a[i]==a[i-1])  
    {  
        cnt++;  
    }  
    else  
    {  
        cnt=1;  
    }  
  
    if(cnt>=3)  
    {  
        // 处理连续段  
    }  
}
```

5. 易错点

连续段题最容易错：

最后一段忘记处理
cnt 初始值写错
i 和 i-1 越界
连续段起点算错

六、二维矩阵模拟

1. 常见模型

二维模拟常见于：

棋盘
地图
矩阵
网格
消除游戏
行列操作

2. 常见变量

```
int n,m;  
int a[105][105];
```

或者：

```
vector<vector<int>> a(n,vector<int>(m));
```

3. 遍历矩阵

```
for(int i=0;i<n;i++)  
{  
    for(int j=0;j<m;j++)  
    {  
        // 处理 a[i][j]  
    }  
}
```

4. 四方向数组

如果要上下左右移动：

```
int dx[4]={-1,1,0,0};
int dy[4]={0,0,-1,1};
```

使用：

```
for(int k=0;k<4;k++)
{
    int nx=x+dx[k];
    int ny=y+dy[k];

    if(nx>=0 && nx<n && ny>=0 && ny<m)
    {
        // 合法位置
    }
}
```

5. 二维模拟的核心

二维模拟最重要的是：

先判断新位置是否合法，再访问数组。

错误写法：

```
if(a[nx][ny]==1 && nx>=0 && nx<n && ny>=0 && ny<m)
```

这样可能已经越界访问。

正确写法：

```
if(nx>=0 && nx<n && ny>=0 && ny<m && a[nx][ny]==1)
```

七、标记数组思想

1. 为什么需要标记数组？

有些题不能边检查边修改。

例如消除游戏：

如果一边检查一边把元素改掉，
可能影响后面的判断。

所以先用标记数组记录：

哪些位置要被修改。

最后统一修改。

2. 模板

```
int mark[105][105]={0};

for(int i=0;i<n;i++)
{
    for(int j=0;j<m;j++)
    {
        if(满足条件)
        {
            mark[i][j]=1;
        }
    }
}

for(int i=0;i<n;i++)
{
    for(int j=0;j<m;j++)
    {
        if(mark[i][j])
        {
            a[i][j]=0;
        }
    }
}
```

3. 适用场景

标记数组适用于：

先判断，后修改
同时发生的变化
不能让前面的修改影响后面的判断

八、日期模拟

1. 日期模拟常见内容

日期模拟通常考：

- 平年 / 闰年
- 每月天数
- 星期偏移
- 日期合法性
- 日期加减

2. 闰年判断

```
bool leap(int y)
{
    if(y%400==0)
    {
        return true;
    }

    if(y%4==0 && y%100!=0)
    {
        return true;
    }

    return false;
}
```

3. 月份天数数组

```
int month[13]={0,31,28,31,30,31,30,31,31,30,31,30,31};
```

如果是闰年：


```
month[2]=29;
```

注意：

每次判断年份时，要重新处理 2 月天数。

4. 日期推进一天

```
d++;  
  
if(d>month[m])  
{  
    d=1;  
    m++;  
}  
  
if(m>12)  
{  
    m=1;  
    y++;  
}
```

5. 日期模拟易错点

闰年规则写错
2 月天数忘记恢复
月份从 1 开始
星期编号和题目不一致
跨年没处理

九、字符串模拟

1. 常见模型

字符串模拟常见于：

字符替换
字符串展开

统计字符
判断回文
格式转换
按规则生成新字符串

2. 字符遍历

```
for(int i=0;i<s.size();i++)
{
    char c=s[i];

    // 处理 c
}
```

3. 判断数字和字母

```
if(c>='0' && c<='9')
{
    // 数字
}

if(c>='a' && c<='z')
{
    // 小写字母
}

if(c>='A' && c<='Z')
{
    // 大写字母
}
```

4. 字符转数字

```
int x=c-'0';
```

5. 数字转字符

```
char c=x+'0';
```

6. 字符串拼接

```
string ans="";  
  
ans+=c;
```

或者：

```
ans.push_back(c);
```

十、编号问题

1. 为什么编号容易错？

题目可能使用：

```
1 号到 n 号
```

但 C++ 数组常用：

```
0 到 n-1
```

这就容易错。

2. 建议

如果题目编号从 1 开始：

```
代码也尽量从 1 开始。
```

例如：

```
for(int i=1;i<=n;i++)  
{
```

```
cin>>a[i];  
}
```

这样更贴合题意。

十一、状态变量设计

1. 什么是状态？

状态就是：

当前模拟到哪里了
当前对象是什么情况
当前还剩什么

2. 常见状态变量

变量	含义
x,y	当前位置
dir	当前方向
cnt	当前计数
flag	是否满足某条件
last	上一个值
now	当前值
ans	答案
t	当前时间

3. 状态更新顺序

模拟题特别容易错在：

先判断还是先更新？
先移动还是先计数？
先修改还是先记录？

建议写代码前先明确：

每一轮循环到底做哪些事，顺序是什么。

十二、输入格式问题

1. 为什么输入格式重要？

很多模拟题不是算法错，而是：

输入读错了。

比如：

有的输入是字符
有的输入是字符串
有的输入带空格
有的输入多组数据

2. 常见输入方式

读整数

```
int x;  
cin>>x;
```

读字符串

```
string s;  
cin>>s;
```

读整行

```
string line;  
getline(cin,line);
```

注意：

如果前面有 `cin>>`，再用 `getline`，可能需要先吃掉换行。

```
cin.ignore();  
getline(cin, line);
```

十三、模拟题调试方法

1. 用小样例手推

不要只看大样例。

自己造：

最小数据
边界数据
特殊数据

2. 打印中间状态

例如：

```
cout<<"i="<<i<<" cnt="<<cnt<<"\n";
```

看程序是否按预期运行。

3. 检查循环范围

重点看：

`i=0` 还是 `i=1`
`i<n` 还是 `i<=n`
`j<m` 还是 `j<=m`

十四、模拟题常见错误

1. 越界访问

错误：

```
a[i+1]
```

但 `i` 已经是最后一个位置。

2. 忘记处理最后一段

连续段统计时：

循环结束后最后一段可能还没处理。

3. 边检查边修改

如果规则要求“同时发生”，通常不能边检查边修改。

要用：

标记数组

4. 编号错位

题目是 1 号到 n 号，代码用 0 到 $n-1$ ，需要特别小心。

5. 更新顺序错

例如：

应该先判断再移动，
却写成先移动再判断。

十五、CSP 模拟题常见类型

类型	常见内容
一维模拟	数组、编号、序列变化
二维模拟	棋盘、矩阵、消除
日期模拟	闰年、星期、月份
字符串模拟	替换、展开、统计
事件模拟	按时间顺序处理
状态机模拟	多种状态之间切换

十六、本专题做过的题

题号 / 题目	类型	题面简述	对应知识点
CSP 201512-1 数位之和	基础模拟	输入一个整数，求各位数字之和	数字拆位
CSP 201512-2 消除类题	二维模拟	判断矩阵中连续相同元素并消除	标记数组
CSP 日期类题	日期模拟	根据年份、月份、星期等规则推日期	闰年、月份天数
P2670 扫雷游戏	二维模拟	根据地雷位置统计周围地雷数	八方向遍历
P1563 玩具谜题	环形模拟	按方向和步数在环上移动	取模、编号
P1098 字符串的展开	字符串模拟	按规则展开字符串中的区间	字符处理
P1203 坏掉的项链	环形模拟	在环形项链中找最长可取连续段	环形处理

十七、本章最重要的结论

模拟题的核心不是高级算法，而是：

规则准确
状态清楚
边界稳定

做模拟题时，先不要急着写代码。

先问自己：

我要模拟哪些对象？
每个对象有哪些状态？
每一步按什么顺序发生？
什么时候结束？
边界在哪里？

如果这些问题想清楚，模拟题就会稳定很多。